

day 9

📌 01. Hoisting & Variable Scopes (var , let , const)

المفهوم الأساسي 💡 📌

Hoisting قبل تنفيذ الكود Scope إلى أعلى الـ (Declarations) لرفع التصريحات JavaScript هو سلوك محرك

يختلف هذا السلوك بين var و (let / const).

1. Global Window Object & Scopes

- **var**: window object تُحفظ كخاصية داخل الـ Global Scope في الـ var عند الإعلان عن:
- **let & const**: لا يُحفظان في الـ window object رغم وجودهما في الـ Global Scope.

JavaScript

```
// Global Scope Check
var globalVar = "Iam global var";
let globalLet = "Iam global let";
const globalConst = "Iam global const";

console.log(window.globalVar); // 🟢 "Iam global var"
console.log(window.globalLet); // 🔴 undefined
console.log(window.globalConst); // 🔴 undefined
```

2. Function Scope vs Block Scope

⚠️ ⚠️ فرق مهم جداً

- **Function Scope**: (var , let , const) المتغير لا يُرى خارج الدالة أي كان نوعه.
- **Block Scope**: { } لا يُرى خارجها إلا var المتغير داخل أقواس { }

◆ Function Scope Example:

JavaScript

```
function sayHello() {
  var x = 10;
```

```

    let y = 20;
    const z = 30;
  }
  sayHello();

// console.log(x); // 🚫 Uncaught ReferenceError: x is not defined
// console.log(y); // 🚫 Uncaught ReferenceError: y is not defined

```

◆ Block Scope Example:

JavaScript

```

if (true) {
  var x = 10; // 🌐 Function/Global Scope
  let y = 20; // 📦 Block Scope
  const z = 30; // 📦 Block Scope
}

console.log(x); // 🟢 10 (var تسربت خارج الـ Block)
// console.log(y); // 🚫 ReferenceError: y is not defined
// console.log(z); // 🚫 ReferenceError: z is not defined

```

📋 جدول المقارنة الشامل (var vs let vs const)

Feature	var	let	const
Global Window Attachment	🟢 Yes	🚫 No	🚫 No
Function Scope	🟢 Yes	🟢 Yes	🟢 Yes
Block Scope	🚫 No	🟢 Yes	🟢 Yes
Can Be Reassigned?	🟢 Yes	🟢 Yes	🚫 No
Can Be Redeclared?	🟢 Yes	🚫 No	🚫 No
Hoisting Behavior	undefined	TDZ (ReferenceError)	TDZ (ReferenceError)

🔗 أسئلة إنترفيو متوقعة (Interview Q&A) ?

🔗 Q1: What is Temporal Dead Zone (TDZ)?

الإجابة: هي المنطقة الزمنية بين بداية الـ Scope وحتى السطر الذي تم فيه الإعلان عن `let` أو `const`. إذا حاولت الوصول للمتغير خلال هذه المنطقة، ستحصلين على `ReferenceError` بعكس `var` التي ترجع `undefined`.

❓ Q2: Why does `var` cause issues in `for` loops with `async` code?

الإجابة: لأن `var` تُعامل كـ Function Scope وليس Block Scope، بالتالي لا يتم إنشاء Scope جديد لكل Iteration، بل تتغير القيمة لنفس المتغير في كل دورة. الحل هو استخدام `let`.

📌 02. Template Literals & Default Parameters

1. Template Literals

تتيح كتابة النصوص متعددة الأسلاب (Multi-line) ودمج المتغيرات بسلاسة باستخدام `${}`.

JavaScript

```
const name = "ali";
const age = 26;

const message = `hello ${name} and age
                 is ${age}`;

console.log(message);
```

2. Default Parameters

تحديد قيم افتراضية لـ Parameters الدالة لتجنب الـ `undefined`.

JavaScript

```
function sayHello(name = "guest") {
  console.log(`hello ${name}`);
}

sayHello("ali");    // hello ali
sayHello();         // hello guest
```

📌 03. Spread & Rest Operators (...)

📌 الفرق السريع

- **Spread Operator:** (Spreads elements) إلى عناصر منفصلة Array/Object بفرد الـ
- **Rest Operator:** (Gathers elements) واحدة Array يجمع العناصر المنفصلة داخل

1. Spread Operator Examples

◆ Cloned vs Referenced Arrays:

JavaScript

```
// Copying Array (Pass by Value / Shallow Copy)
let arr1 = [1, 2, 3];
let arr2 = [...arr1]; // إنشاء مصفوفة جديدة في مكان جديد في الـ Memory

arr1 = [5, 6, 7];
console.log(arr2); // [1, 2, 3] (لم تتأثر)

// Pointer Reference (Pass by Reference)
let arr6 = [1, 2, 3];
let arr7 = arr6; // Memory يشار إلى نفس المكان في الـ Memory
arr6.push(6);
console.log(arr7); // [1, 2, 3, 6] (تأثرت بالتعديل)
```

◆ Objects & Strings:

JavaScript

```
// Merging & Overriding Objects
let obj1 = { a: 1, b: 2 };
let obj2 = { ...obj1, b: 3 }; // إلى 3 b يتم تعديل
console.log(obj2); // { a: 1, b: 3 }

// Converting String to Array of Characters
let str = "hello";
let chars = [...str];
console.log(chars); // ['h', 'e', 'l', 'l', 'o']
```

2. Rest Operator Examples

يستخدم داخل معلمات الدالة (Function Arguments) لتجميع أي عدد غير محدد من المدخلات:

JavaScript

```
function sum(...args) {
  // args تحتوي كل القيم الممررة Array عبارة عن
  return args.reduce((acc, cur) => acc + cur, 0);
}
```

```
console.log(sum()); // ● 0
console.log(sum(5)); // ● 5
console.log(sum(5, 5, 5)); // ● 15
```

📌 04. Arrow Functions

صيغة مختصرة لكتابة الدوال في ES6 تتميز بعدم وجود `this` خاص بها (تأخذ `this` من الـ Lexical Scope).

JavaScript

```
// One-liner implicit return
const sum = (a, b) => a + b;
console.log(sum(5, 5)); // ● 10

// Multi-line block with object return
let calc = (a, b) => {
  const sum = a + b;
  const diff = a - b;
  return { sum, diff };
};
console.log(calc(5, 5)); // ● { sum: 10, diff: 0 }
```

📌 05. Array High Order Functions & Native Implementation

📌 i مفهوم الدالة النقية (Pure Implementation)

أسئلة المقابلات واختبارات الـ Problem Solving تطلب دائماً إعادة كتابة دوال مثل (`map`, `filter`, `reduce`) بدون استخدام الميثود المدمجة في JS.

● 1. Map (`map`)

ترجع **Array جديدة** بنفس طول `Array` الأصلية بعد تطبيق دالة على كل عنصر.

JavaScript

```
// Built-in Usage
const names = ["ali", "ahmed", "mohamed"];
const newNames = names.map((name, i) => `${i} - ${name} salem`);
console.log(newNames);

// 🦄 Pure JS Native Implementation
```

```

var mapNative = function (arr, fn) {
  var newArray = [];
  for (let i = 0; i < arr.length; i++) {
    newArray.push(fn(arr[i], i));
  }
  return newArray;
};

const NativeNames = mapNative(names, (name, i) => `${i} - ${name} salem`);
console.log(NativeNames);

```

2. Filter (filter)

ترجع **Array جديدة** تحتوي فقط على العناصر التي حققت الشرط (true).

JavaScript

```

// Built-in Usage
const names = ["ali", "ahmed", "mohamed"];
const filteredNames = names.filter((name) => name.startsWith("a"));
console.log(filteredNames); // ● ["ali", "ahmed"]

// 🛠 Pure JS Native Implementation
var filterNative = function (arr, fn) {
  var newArr = [];
  for (let i = 0; i < arr.length; i++) {
    if (fn(arr[i], i)) {
      newArr.push(arr[i]);
    }
  }
  return newArr;
};

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9];
const evenNumbers = filterNative(numbers, (number) => number % 2 === 0);
console.log(evenNumbers); // ● [2, 4, 6, 8]

```

3. Reduce (reduce)

تختزل عناصر الـ **Array** في **قيمة واحدة فقط** (Single Value) سواء كانت رقم، نص، أو **Object**.

تتبع عمل الـ **Accumulator** 🇸🇩

Plaintext

acc (Accumulator)	cur (Current)	acc + cur (Total)
0	1	1
1	2	3
3	3	6
6	4	10
10	5	15 --> Result

JavaScript

```
// Built-in Usage
const numbers = [1, 2, 3, 4, 5];
const totalSum = numbers.reduce((accumulator, current) => accumulator +
current, 0);
console.log(totalSum); // 15
```

```
// 🦄 Pure JS Native Implementation
var reduceNative = function (arr, fn, initialValue) {
  let value = initialValue;
  for (let i = 0; i < arr.length; i++) {
    value = fn(value, arr[i], i);
  }
  return value;
};
```

```
const nativeSum = reduceNative(numbers, (acc, cur) => acc + cur, 0);
console.log(nativeSum); // 15
```

? أسئلة إنترفيو متوقعة على الـ Array Methods

? Q1: Does `map()` mutate the original array?

الإجابة: لا، `map` لا تعدل المصفوفة الأصلية وإنما ترجع مصفوفة جديدة (Immutability).

? Q2: What happens if you don't provide an initial value to `reduce()` ?

الإجابة: سيتخذ `reduce` العنصر الأول في المصفوفة كـ `accumulator` افتراضي، ويبدأ الـ Loop من العنصر الثاني (Index 1). وفي حالة كانت المصفوفة فارغة سيعطي `TypeError`.